Entregable 3 · Mi primer dashboard (Olist)

Dave Jairo Carvajal Gonzalez · UPSE · 6/1 · Oct 25, 2026 · @Dave

1. Dataset y comprensión del problema

El proyecto analiza 98,666 pedidos reales del marketplace brasileño Olist, entre septiembre de 2016 y septiembre de 2018, para integrar ventas, logística y satisfacción del cliente en un solo dashboard.

Campo	Contenido
Nombre	Brazilian E-Commerce Public Dataset by Olist
Enlace	kaggle.com/olistbr/brazilian-ecommerce
Descripción	Pedidos anonimizados con estado, precio, flete, pago, ubicación del cliente, producto, vendedor y reseña. Cubre el ciclo completo: compra, pago, envío, entrega y opinión.
Archivos	9 CSV; se usan 8 y se descarta `geolocation` (1,000,163 filas, muy duplicadas y sin aporte a los KPI).
Período	Compras del 4/09/2016 al 17/10/2018 en `orders`. El modelo llega hasta el 3/09/2018, porque solo incluye pedidos con ítems.
Registros	orders 99,441 · order_items 112,650 · order_payments 103,886 · order_reviews 99,224 · customers 99,441 (96,096 únicos) · products 32,951 · sellers 3,095

Qué representa una fila. En `orders`, un pedido. En `order_items`, un producto dentro de un pedido (clave `order_id` + `order_item_id`). En `order_payments`, un pago o cuota. En `order_reviews`, una reseña. En `customers`, un `customer_id` por pedido; el cliente real se identifica con `customer_unique_id`.

Variables principales.

- Identificadores: `order_id`, `customer_unique_id`, `product_id`, `seller_id`.
- Numéricas: `price`, `freight_value`, `payment_value`, `review_score` (1 a 5), peso y dimensiones del producto.
- Categóricas: categoría del producto, tipo de pago, estado y ciudad.
- Fechas: compra, aprobación, envío, entrega real y entrega estimada.

Problemas de calidad identificados (cifras del análisis exploratorio).

- Fechas almacenadas como texto.
- 2,965 pedidos sin fecha de entrega y 160 sin fecha de aprobación.
- 547 pedidos con más de una reseña.
- 610 productos sin categoría.
- 3 pagos con tipo `not_defined`.

- Categorías en portugués y ciudades con grafías distintas.
- Entregas anteriores a la compra: 0 (se verificó que no existen).

Justificación. El dataset tiene relaciones claras entre tablas, varias fechas que permiten medir tiempos y dimensiones de producto, vendedor y geografía. Eso permite un esquema en estrella con KPI de tres naturalezas: ventas, logística y satisfacción. Su volumen es suficiente para un análisis real y manejable en un PostgreSQL local.

Problema de análisis. La gerencia no ve de forma integrada cómo se comportan las ventas, qué tan bien funciona la entrega y si los retrasos afectan la satisfacción del cliente.

Usuarios y decisiones.

- Gerencia comercial: qué categorías y regiones impulsar y cuándo hacer campañas.
- Logística: en qué estados renegociar con transportistas y cómo ajustar los plazos estimados.
- Atención al cliente: qué categorías necesitan seguimiento por bajas calificaciones.

2. Preguntas de negocio e indicadores

Tres preguntas de negocio guían el dashboard, y cada KPI se calcula con una función SQL sobre el modelo dimensional implementado.

#	Pregunta de negocio	Objeto SQL que la responde
P1	¿Cómo evolucionan las ventas y los pedidos por mes, y qué categorías generan más ingresos?	`fn_ventas_por_mes`, `fn_top_categorias`
P2	¿Qué tan eficiente es la logística: cuánto tarda la entrega, qué porcentaje llega tarde y en qué estados es peor?	`fn_tiempo_promedio_entrega`, `fn_tasa_entregas_tardias`, `fn_tardias_por_estado`
P3	¿El retraso en la entrega reduce la satisfacción del cliente?	`fn_satisfaccion_vs_retraso`

KPI	Fórmula	Medida del modelo	Pregunta
Ingresos totales	Σ `precio` de ventas válidas	`fact_ventas.precio`	P1
Ticket promedio	Σ `precio` ÷ pedidos distintos	`precio`, `order_id`	P1
% entregas tardías	Pedidos entregados después de la fecha estimada ÷ pedidos entregados	`entregado_tarde`	P2
Tiempo promedio de entrega	Promedio por pedido de días entre compra y entrega	`dias_entrega`	P2
Satisfacción promedio	Promedio por pedido de `review_score`	`review_score`	P3

KPI	Fórmula	Medida del modelo	Pregunta
Tasa de cancelación	Pedidos `canceled` ÷ pedidos con ítems	`dim_estado_pedido.order_status`	Apoyo

Regla de negocio. Una venta válida excluye los pedidos `canceled` y `unavailable`; la regla vive en `dim_estado_pedido.es_venta_valida`.

Precisión sobre la granularidad. Como la tabla de hechos es por ítem, `review_score` y `dias_entrega` se repiten en cada ítem de un pedido. Por eso las funciones los toman una vez por pedido y usan `COUNT(DISTINCT order_id)`, para que los pedidos con varios productos no pesen más.

3. Limpieza y transformación de datos

La limpieza se hace con Python y pandas en `02_etl_cargar.py`, antes de cargar el modelo dimensional; el script vacía las tablas al inicio, así que puede ejecutarse varias veces.

Tabla origen	Problema encontrado	Acción aplicada
orders	Fechas como texto	Conversión con `pd.to_datetime`; se eliminan `order_id` duplicados.
orders	2,965 pedidos sin fecha de entrega	Se conservan como NULL, porque indican pedidos no entregados.
orders	Entregas anteriores a la compra	Se verificó que hay 0; el script igualmente las pasaría a NULL.
order_items	Posibles duplicados e importes inválidos	Se eliminan duplicados por (`order_id`, `order_item_id`) y filas con `price` o `freight_value` negativos o nulos.
order_payments	Varios pagos por pedido; 3 `not_defined`; cuotas en 0	Un registro por pedido: tipo de mayor valor y cuotas máximas agrupadas en rangos. `not_defined` pasa a `no_definido` y cuotas 0 pasan a 1.
order_reviews	547 pedidos con más de una reseña; comentarios casi vacíos	Se conserva la reseña más reciente por pedido y se descartan los textos libres.
products	610 sin categoría; nombres en portugués; columnas con error ortográfico	Nulos pasan a `sin_categoria`; se traduce con el archivo de traducción (y dos categorías agregadas a mano); se calcula `volumen_cm3` y se descartan las columnas de longitud.
customers	`customer_id` distinto por pedido; ciudades con grafías distintas	Se usa `customer_unique_id` como clave natural (96,096 clientes), con su ubicación más reciente; ciudades en minúsculas, sin acentos ni espacios sobrantes.

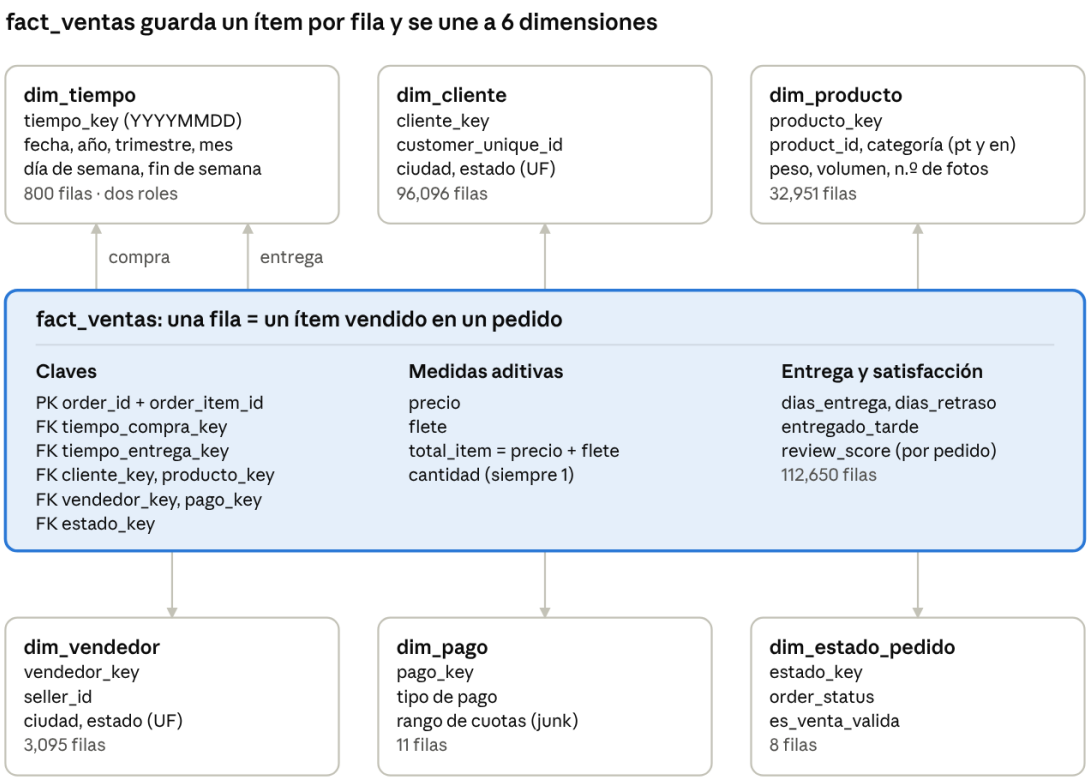
Tabla origen	Problema encontrado	Acción aplicada
sellers	Ciudades con grafías distintas	Misma normalización de texto que en clientes.
geolocation	1,000,163 filas con muchos duplicados	Se descarta del modelo; se conserva el estado (UF) en clientes y vendedores.
Valores atípicos	Precios, fletes y tiempos de entrega extremos	No se eliminan, porque son ventas reales; quedan disponibles para analizarlos con filtros.

Variables derivadas. `total_item` (precio más flete), `dias_entrega` (entrega real menos compra), `dias_retraso` (entrega real menos estimada), `entregado_tarde` (retraso mayor a 0) y `volumen_cm3`.

Pedidos sin ítems. La tabla de hechos se construye desde `order_items`, por lo que 775 pedidos de `orders` sin ítems no entran al modelo (el 99,441 original queda en 98,666). Son sobre todo cancelados, no disponibles y recién creados.

4. Modelo dimensional

Granularidad. Una fila de `fact_ventas` representa un ítem (un producto) vendido dentro de un pedido de Olist; su clave primaria es (`order_id`, `order_item_id`).



esquema en estrella · 1 tabla de hechos, 6 dimensiones

La tabla de hechos se une a seis dimensiones; `dim_tiempo` participa dos veces, como fecha de compra y como fecha de entrega (dimensión con roles).

Se eligió el ítem y no el pedido porque un pedido puede tener varios productos de distintas categorías y vendedores, y así se analiza por categoría y por vendedor sin perder precisión. Los datos de pedido (estado, entrega, reseña, pago) se repiten en cada ítem, y por eso los KPI de pedido usan `COUNT(DISTINCT order_id)` o un valor por pedido.

Dimensión	Clave primaria	Clave natural	Qué describe
dim_tiempo	`tiempo_key` (YYYYMMDD)	`fecha`	Calendario continuo con año, trimestre, mes y día de la semana
dim_cliente	`cliente_key`	`customer_unique_id`	Cliente real, con ciudad y estado
dim_producto	`producto_key`	`product_id`	Categoría original y traducida, peso, volumen y fotos
dim_vendedor	`vendedor_key`	`seller_id`	Vendedor, con ciudad y estado
dim_pago	`pago_key`	(`tipo_pago`, `rango_cuotas`)	Dimensión junk que combina tipo de pago y rango de cuotas
dim_estado_pedido	`estado_key`	`order_status`	Estado y bandera `es_venta_valida`

Medida	Tipo	KPI que alimenta
`precio`	Aditiva	Ingresos totales, ticket promedio
`flete`	Aditiva	Análisis de costo logístico
`total_item`	Aditiva	Ingreso bruto con envío
`cantidad`	Aditiva (siempre 1)	Conteo de ítems vendidos
`dias_entrega`	Semiaditiva (se promedia)	Tiempo promedio de entrega
`dias_retraso`	Semiaditiva (se promedia)	Magnitud del retraso
`entregado_tarde`	Indicador	% de entregas tardías
`review_score`	No aditiva (a nivel de pedido)	Satisfacción promedio

5. Implementación en PostgreSQL

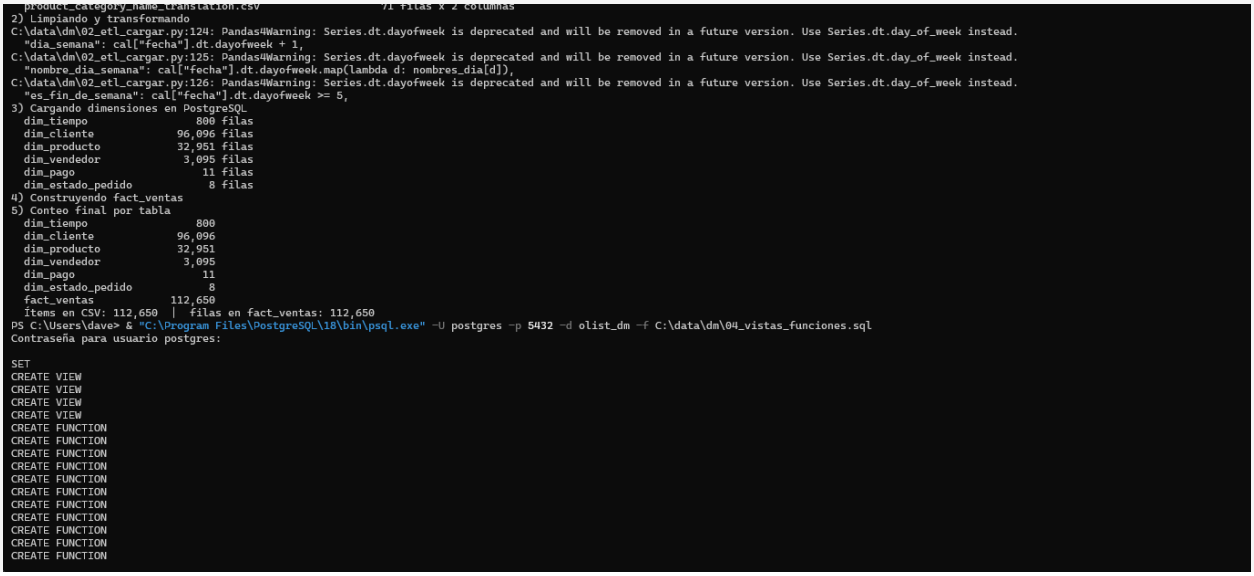
El modelo vive en la base `olist_dm`, esquema `dm`, sobre PostgreSQL 18 (puerto 5432). Se carga con un proceso reproducible y se consulta solo mediante vistas y funciones.

Archivo	Qué hace
`01_crear_tablas.sql`	Crea el esquema, las 6 dimensiones y la tabla de hechos, con claves primarias, claves foráneas, restricciones CHECK e índices.
`02_etl_cargar.py`	Lee los CSV, limpia, carga las dimensiones, construye `fact_ventas` con claves sustitutas e imprime los conteos finales.
`03_validaciones.sql`	Consultas de validación: registros, integridad, nulos y totales.
`04_vistas_funciones.sql`	6 vistas y 11 funciones que consume la aplicación.

Integridad referencial. `fact_ventas` tiene clave primaria (`order_id`, `order_item_id`) y 7 claves foráneas a las dimensiones; la carga usa `inner join` contra cada dimensión, así que ninguna fila queda sin su dimensión.

Ejecución (PowerShell).

```
psql -U postgres -p 5432 -c "CREATE DATABASE olist_dm;"
psql -U postgres -p 5432 -d olist_dm -f 01_crear_tablas.sql
$env:OLIST_DIR = "C:\data\olist"
$env:DB_URL = "postgresql+psycopg2://postgres:CLAVE@localhost:5432/olist_dm"
py 02_etl_cargar.py
psql -U postgres -p 5432 -d olist_dm -f 04_vistas_funciones.sql
psql -U postgres -p 5432 -d olist_dm -c "ALTER DATABASE olist_dm SET search_path TO dm, public;"
psql -U postgres -p 5432 -d olist_dm -f 03_validaciones.sql
```



```
CREATE VIEW
PS C:\Users\dave> & "C:\Program Files\PostgreSQL\18\bin\psql.exe" -U postgres -p 5432 -d olist_dm -f C:\data\dm\03_validaciones.sql -o C:\data\dm\salida_validaciones.txt
Contraseña para usuario postgres:

PS C:\Users\dave> notepad C:\data\dm\salida_validaciones.txt
PS C:\Users\dave> & "C:\Program Files\PostgreSQL\18\bin\psql.exe" -U postgres -p 5432 -d olist_dm -c "SELECT dm.fn_total_ingresos(), dm.fn_ticket_promedio(), dm.fn_tasa_entregas_tardias(), dm.fn_satisfacci
on_promedio(), dm.fn_tasa_cancelacion();"
Contraseña para usuario postgres:

ERROR:  no existe la relación «vw_ventas_detalle»
LÍNEA 3:     FROM vw_ventas_detalle
                  ^
CONSULTA:
    SELECT COALESCE(SUM(precio), 0)
    FROM vw_ventas_detalle
    WHERE es_venta_valida
      AND (p_ini IS NULL OR fecha_compra >= p_ini)
      AND (p_fin IS NULL OR fecha_compra <= p_fin)
      AND (p_categoria IS NULL OR categoria = p_categoria)
      AND (p_estado_cliente IS NULL OR estado_cliente = p_estado_cliente);

CONTEXTO:  función SQL «fn_total_ingresos», durante expansión en línea
PS C:\Users\dave> & "C:\Program Files\PostgreSQL\18\bin\psql.exe" -U postgres -p 5432 -d olist_dm -f "C:\data\dm\04_vistas_funciones.sql"
Contraseña para usuario postgres:

SET
CREATE VIEW
CREATE VIEW
CREATE VIEW
CREATE VIEW
CREATE FUNCTION
CREATE FUNCTION
CREATE FUNCTION
CREATE FUNCTION
CREATE FUNCTION
CREATE FUNCTION
CREATE FUNCTION
CREATE FUNCTION
CREATE FUNCTION
CREATE FUNCTION
CREATE FUNCTION
CREATE VIEW
CREATE VIEW
PS C:\Users\dave> & "C:\Program Files\PostgreSQL\18\bin\psql.exe" -U postgres -p 5432 -d olist_dm -c "ALTER DATABASE olist_dm SET search_path TO dm, public;"
CREATE VIEW
PS C:\Users\dave> & "C:\Program Files\PostgreSQL\18\bin\psql.exe" -U postgres -p 5432 -d olist_dm -c "ALTER DATABASE olist_dm SET search_path TO dm, public;"
Contraseña para usuario postgres:

ALTER DATABASE
PS C:\Users\dave> & "C:\Program Files\PostgreSQL\18\bin\psql.exe" -U postgres -p 5432 -d olist_dm -c "SELECT dm.fn_total_ingresos(), dm.fn_ticket_promedio(), dm.fn_tasa_entregas_tardias(), dm.fn_satisfacci
on_promedio(), dm.fn_tasa_cancelacion();"
Contraseña para usuario postgres:

 fn_total_ingresos | fn_ticket_promedio | fn_tasa_entregas_tardias | fn_satisfaccion_promedio | fn_tasa_cancelacion
-------------------+--------------------+--------------------------+--------------------------+---------------------
       13494400.74 |             137.42 |                     6.77 |                     4.12 |                0.47
(1 fila)


PS C:\Users\dave> py -c "import pandas as pd; i=pd.read_csv(r'C:\data\olist\olist_order_items_dataset.csv'); print(round(i.price.sum(),2), round(i.freight_value.sum(),2))"
13591643.7 2251909.54
PS C:\Users\dave> mkdir C:\data\app


    Directorio: C:\data


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        25/10/2026     15:10                app


PS C:\Users\dave> Move-Item "$env:USERPROFILE\Downloads\app.py" C:\data\app\
PS C:\Users\dave> $env:DB_URL = "postgresql+psycopg2://postgres:123@localhost:5432/olist_dm"
PS C:\Users\dave> cd C:\data\app
PS C:\data\app> py -m streamlit run app.py

      👋 Welcome to Streamlit!

      If you'd like to receive helpful onboarding emails, news, offers, promotions,
      and the occasional swag, please enter your email address below. Otherwise,
      leave this field blank.

      Email: dajacago19882@gmail.com

  You can find our privacy policy at https://streamlit.io/privacy-policy

  Summary:

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        25/10/2026     15:10                app


PS C:\Users\dave> Move-Item "$env:USERPROFILE\Downloads\app.py" C:\data\app\
PS C:\Users\dave> $env:DB_URL = "postgresql+psycopg2://postgres:123@localhost:5432/olist_dm"
PS C:\Users\dave> cd C:\data\app
PS C:\data\app> py -m streamlit run app.py

      👋 Welcome to Streamlit!

      If you'd like to receive helpful onboarding emails, news, offers, promotions,
      and the occasional swag, please enter your email address below. Otherwise,
      leave this field blank.

      Email: dajacago19882@gmail.com

  You can find our privacy policy at https://streamlit.io/privacy-policy

  Summary:
  - This open source library collects usage statistics.
  - We cannot see and do not store information contained inside Streamlit apps,
    such as text, charts, images, etc.
  - Telemetry data is stored in servers in the United States.
  - If you'd like to opt out, add the following to %userprofile%/.streamlit/config.toml,
    creating that file if necessary:

    [browser]
    gatherUsageStats = false

2026-10-25 15:10:43.493 Uvicorn server started on :::8501

  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8501
  Network URL: http://192.168.0.3:8501

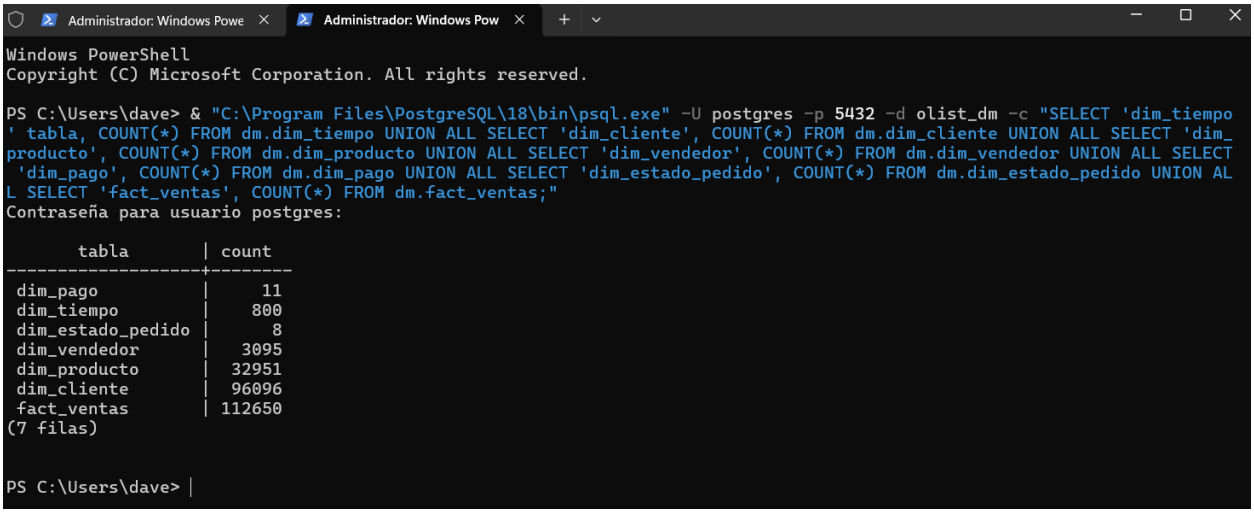
  Help agents write better Streamlit apps?
  Install the official Streamlit skills by running streamlit skills in your terminal.
```

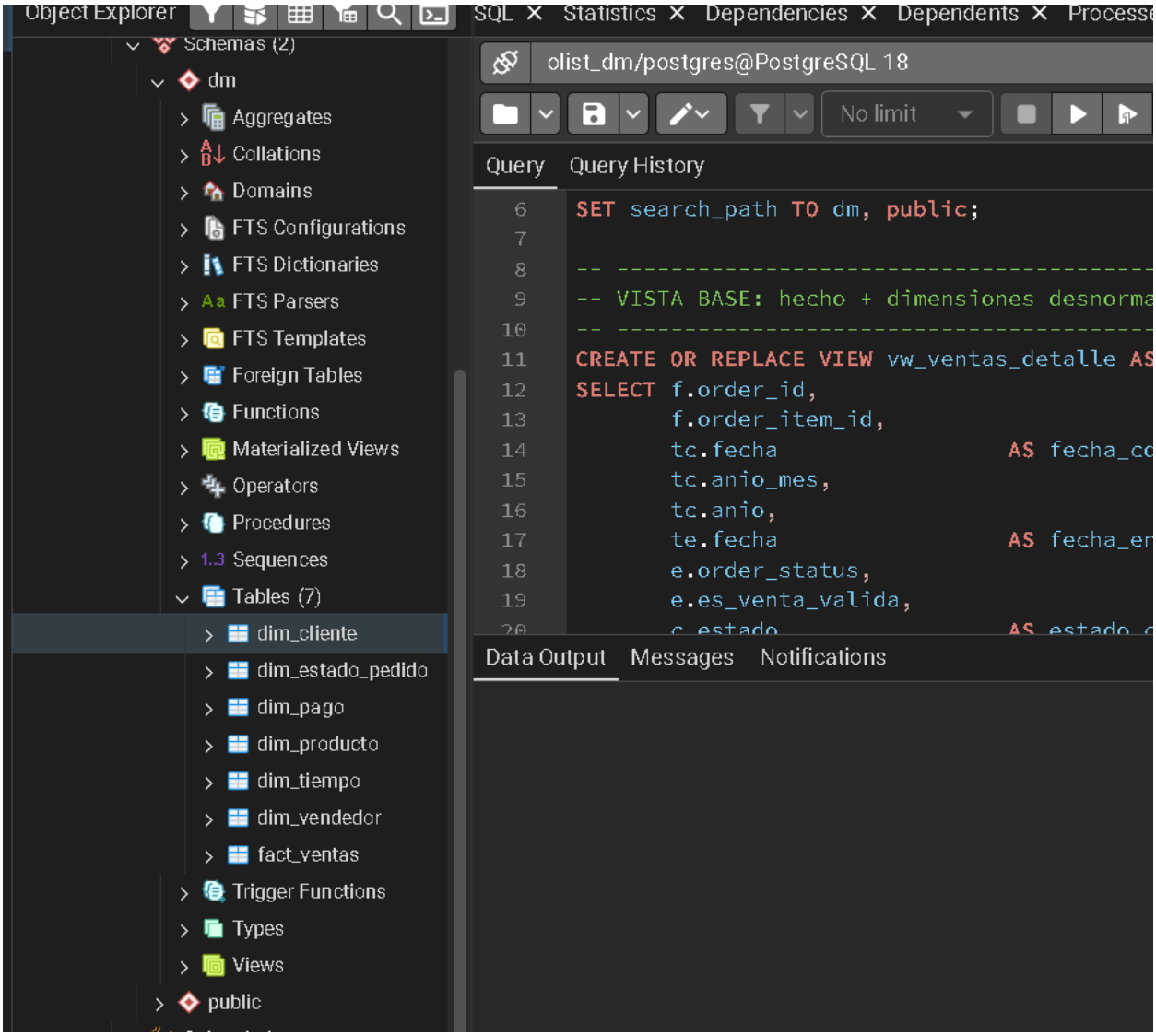
Detalle importante. Las funciones consultan `vw_ventas_detalle` sin indicar el esquema. Por eso se fija `search_path` en la base: sin ese ajuste, una conexión nueva (como la de Streamlit) no encuentra la vista.

Resultado de la carga (salida real del ETL).

Tabla	Filas
dim_tiempo	800
dim_cliente	96,096
dim_producto	32,951
dim_vendedor	3,095
dim_pago	11
dim_estado_pedido	8
fact_ventas	112,650

Captura: salida del ETL o tablas en pgAdmin.


Administrador: Windows Powe
Administrador: Windows Pow
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
PS C:\Users\dave> & "C:\Program Files\PostgreSQL\18\bin\psql.exe" -U postgres -p 5432 -d olist_dm -c "SELECT 'dim_tiempo' tabla, COUNT(*) FROM dm.dim_tiempo UNION ALL SELECT 'dim_cliente', COUNT(*) FROM dm.dim_cliente UNION ALL SELECT 'dim_producto', COUNT(*) FROM dm.dim_producto UNION ALL SELECT 'dim_vendedor', COUNT(*) FROM dm.dim_vendedor UNION ALL SELECT 'dim_pago', COUNT(*) FROM dm.dim_pago UNION ALL SELECT 'dim_estado_pedido', COUNT(*) FROM dm.dim_estado_pedido UNION ALL SELECT 'fact_ventas', COUNT(*) FROM dm.fact_ventas;"
Contraseña para usuario postgres:
tabla | count
dim_pago | 11
dim_tiempo | 800
dim_estado_pedido | 8
dim_vendedor | 3095
dim_producto | 32951
dim_cliente | 96096
fact_ventas | 112650
(7 filas)
PS C:\Users\dave>


Object Explorer
SQL
Statistics
Dependencies
Dependents
Schemas (2)
dm
Aggregates
Collations
Domains
FTS Configurations
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Views
Operators
Procedures
Sequences
Tables (7)
dim_cliente
dim_estado_pedido
dim_pago
dim_producto
dim_tiempo
dim_vendedor
fact_ventas
Trigger Functions
Types
Views
public
olist_dm/postgres@PostgreSQL 18
No limit
Query
Query History
SET search_path TO dm, public;
-- VISTA BASE: hecho + dimensiones desnorma
CREATE OR REPLACE VIEW vw_ventas_detalle AS
SELECT f.order_id,
f.order_item_id,
tc.fecha AS fecha_co
tc.anio_mes,
tc.anio,
te.fecha AS fecha_en
e.order_status,
e.es_venta_valida,
Data Output
Messages
Notifications

6. Validación de los datos

La carga es correcta: los 10 controles de integridad dieron 0 y los totales cargados coinciden con los del CSV original.

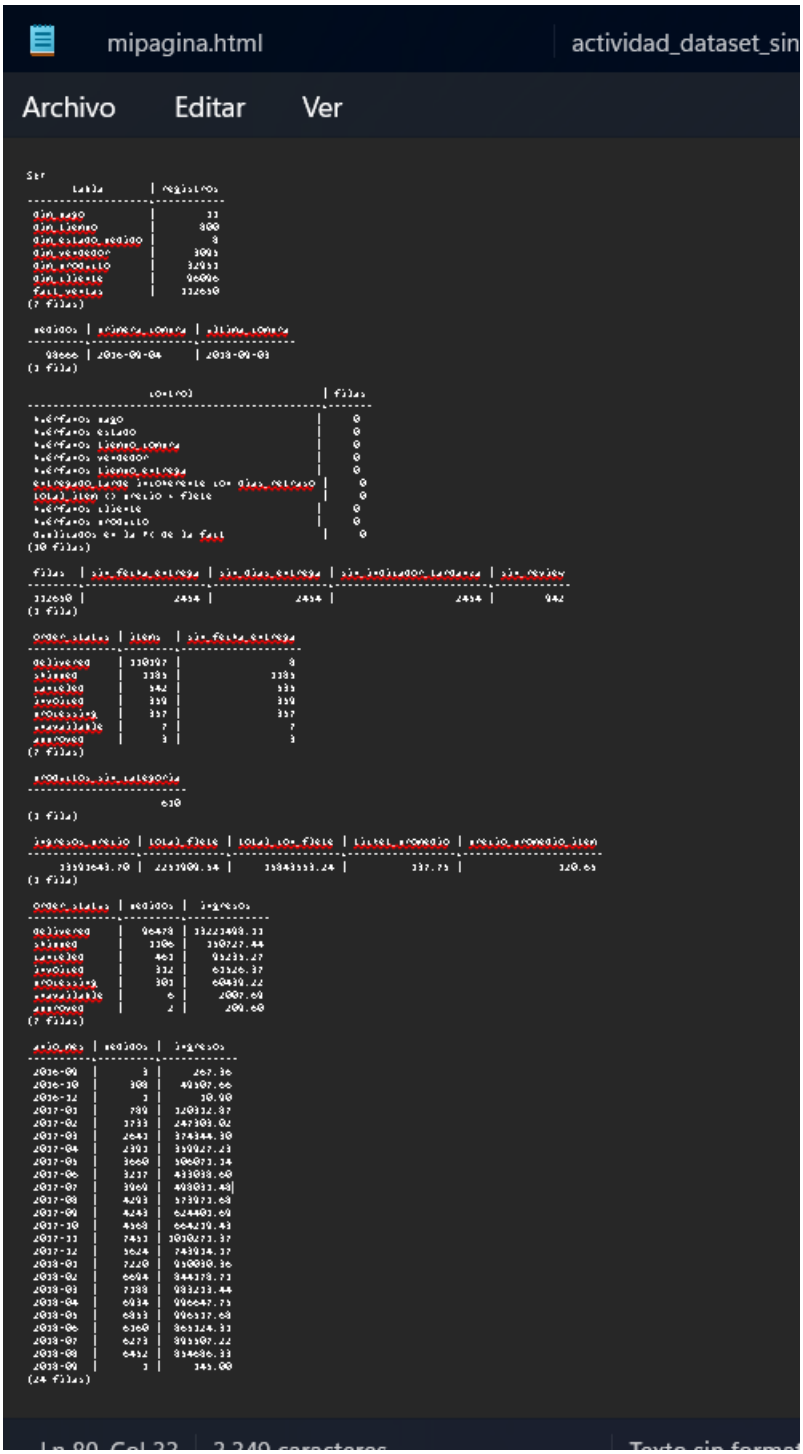
Control	Resultado
Ítems en CSV vs filas en `fact_ventas`	112,650 = 112,650
Suma de `price` en CSV vs `fact_ventas.precio`	13,591,643.70 = 13,591,643.70
Suma de `freight_value` en CSV vs `fact_ventas.flete`	2,251,909.54 = 2,251,909.54
Huérfanos en las 7 claves foráneas	0
Duplicados en la clave primaria de la fact	0
`total_item` distinto de `precio` + `flete`	0
`entregado_tarde` incoherente con `dias_retraso`	0
Pedidos distintos en el modelo	98,666
Suma de pedidos por estado	98,666
Suma de ingresos por estado	13,591,643.70

Nulos relevantes. De 112,650 ítems, 2,454 no tienen fecha de entrega, días de entrega ni indicador de tardanza; corresponden a pedidos no entregados, salvo 8 ítems `delivered` cuya fecha falta en el dato original. Hay 942 ítems sin reseña y 610 productos sin categoría.

Totales principales. Ingresos 13,591,643.70; flete 2,251,909.54; ticket promedio 137.75 (todos los estados). Con la regla de venta válida, los ingresos bajan a 13,494,400.74 (se restan cancelados 95,235.27 y no disponibles 2,007.69).

Hallazgo sobre cancelaciones. El CSV tiene 625 pedidos cancelados (0.63%), pero el modelo solo 461 (0.47%): 164 cancelados no tienen ítems y la fact es por ítem. La tasa de cancelación del dashboard se calcula sobre pedidos con ítems.

Captura: resultado de `03_validaciones.sql` (controles en 0 y totales).


mipagina.html
actividad_dataset_sin
Archivo
Editar
Ver

```
SET
       tabla       | registros
-------------------+-----------
 dim_pago          |        11
 dim_tiempo        |       800
 dim_estado_pedido |         8
 dim_vendedor      |      3095
 dim_producto      |     32951
 dim_cliente       |     96096
 fact_ventas       |    112650
(7 filas)

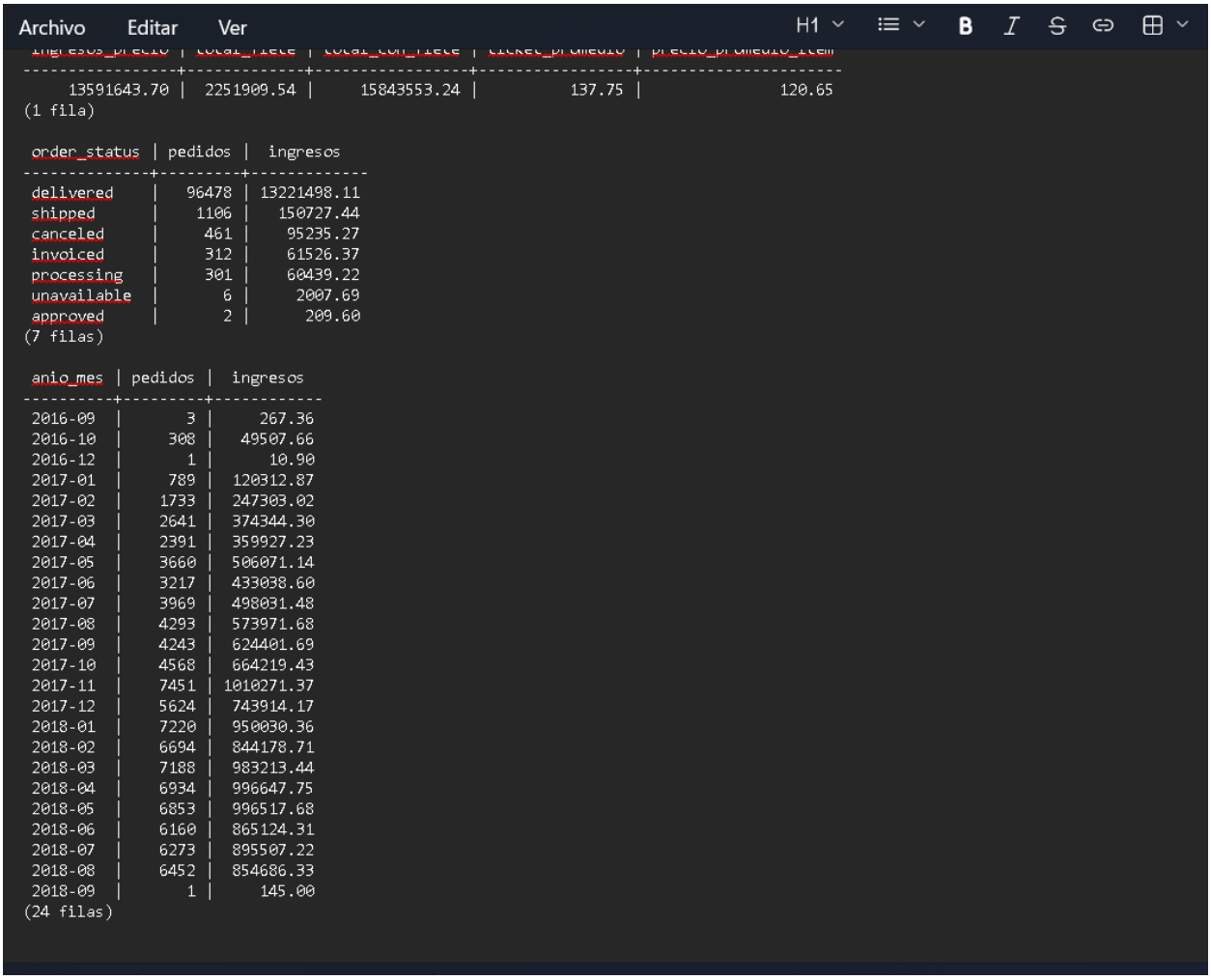
 pedidos | primera_compra | ultima_compra
---------+----------------+---------------
   98666 | 2016-09-04     | 2018-09-03
(1 fila)

                  control                    | filas
---------------------------------------------+-------
 huérfanos pago                              |     0
 huérfanos estado                            |     0
 huérfanos tiempo_compra                     |     0
 huérfanos vendedor                          |     0
 huérfanos tiempo_entrega                    |     0
 entregado_tarde incoherente con dias_retraso |     0
 total_item <> precio + flete                |     0
 huérfanos cliente                           |     0
 huérfanos producto                          |     0
 duplicados en la PK de la fact              |     0
(10 filas)

 filas  | sin_fecha_entrega | sin_dias_entrega | sin_indicador_tardanza | sin_review
--------+-------------------+------------------+------------------------+------------
 112650 |              2454 |             2454 |                   2454 |        942
(1 fila)

 order_status | items  | sin_fecha_entrega
--------------+--------+-------------------
 delivered    | 110197 |                 8
 shipped      |   1185 |              1185
 canceled     |    542 |               535
 invoiced     |    359 |               359
 processing   |    357 |               357
```

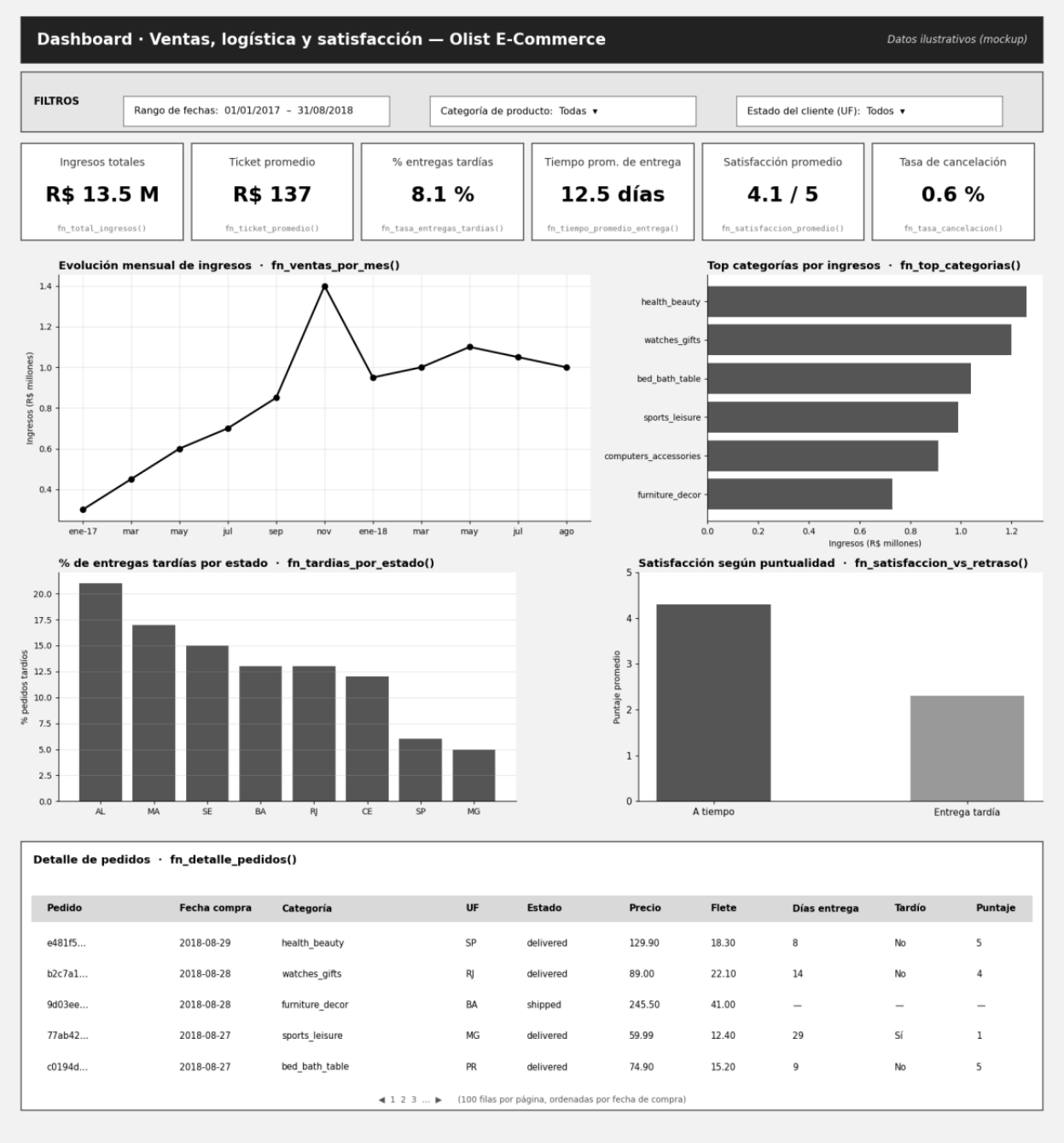
Ln 80, Col 33 | 3.349 caracteres. | Texto sin formato



7. Mockup de la aplicación

El mockup del Entregable 2 se mantiene, porque no hubo retroalimentación con correcciones, y la aplicación final lo sigue con dos ajustes.

Mockup original (Entregable 2).



Elemento	Mockup	Aplicación implementada
Estructura	Una sola página	Dos secciones: Ventas, y Logística y satisfacción
Tarjetas de KPI	6 en una fila	3 por sección (6 en total)
Gráficos	4	4 (2 por sección)
Filtros	Fechas, categoría, estado	Los mismos tres, en la barra lateral

Elemento	Mockup	Aplicación implementada
Tabla de detalle	Sí	Sí, con selector de filas y botón de descarga CSV

8. Matriz de trazabilidad

Cada componente de la aplicación responde a una pregunta de negocio y consume un objeto SQL concreto; todos los datos salen de `fact_ventas` y sus dimensiones.

Componente	Sección	Pregunta / KPI	Filtros	Datos utilizados	Vista / función SQL
Tarjeta Ingresos totales	Ventas	P1	Fecha, categoría, UF	`precio`, `es_venta_valida`	`fn_total_ingresos()`
Tarjeta Ticket promedio	Ventas	P1	Fecha, categoría, UF	`precio`, `order_id`	`fn_ticket_promedio()`
Tarjeta Tasa de cancelación	Ventas	Apoyo	Fecha, categoría, UF	`order_status`	`fn_tasa_cancelacion()`
Gráfico de líneas, ingresos por mes	Ventas	P1	Fecha, categoría, UF	`dim_tiempo.anio_mes`, `precio`	`fn_ventas_por_mes()`
Gráfico de barras, top 10 categorías	Ventas	P1	Fecha, UF	`dim_producto.categoria_en`, `precio`	`fn_top_categorias()`
Tarjeta Entregas tardías	Logística	P2	Fecha, categoría, UF	`entregado_tarde`	`fn_tasa_entregas_tardias()`
Tarjeta Tiempo de entrega	Logística	P2	Fecha, categoría, UF	`dias_entrega`	`fn_tiempo_promedio_entrega()`
Tarjeta Satisfacción	Logística	P3	Fecha, categoría, UF	`review_score`	`fn_satisfaccion_promedio()`
Gráfico de barras, % tardías por estado	Logística	P2	Fecha, categoría	`dim_cliente.estado`, `entregado_ta`	`fn_tardias_por_estado()`

Componente	Sección	Pregunta / KPI	Filtros	Datos utilizados	Vista / función SQL
				`rde`	
Gráfico de barras, satisfacción según puntualidad	Logística	P3	Fecha, categoría, UF	`entregado_tarde`, `review_score`	`fn_satisfaccion_vs_retraso()`
Tabla de detalle	Logística	Todas	Fecha, categoría, UF	`vw_ventas_detalle`	`fn_detalle_pedidos()`
Listas de filtros	Barra lateral	—	—	`dim_producto`, `dim_cliente`, `dim_tiempo`	`vw_filtro_categorias`, `vw_filtro_estados_cliente`, `MIN/MAX(fecha_compra)`

Dos gráficos comparan grupos y por eso no aplican todos los filtros: el top de categorías ignora el filtro de categoría y el de estados ignora el filtro de estado.

9. Aplicación Python + Streamlit

La aplicación (`app.py`) se conecta a PostgreSQL con SQLAlchemy y psycopg2 y obtiene todos los datos de las funciones `dm.fn_*`; no lee ningún CSV. Cumple los mínimos: 2 secciones, 6 KPI, 3 filtros que cambian las consultas, 4 gráficos y 1 tabla de detalle.

Cómo funciona.

- La cadena de conexión se lee de la variable de entorno `DB_URL`, para no escribir la contraseña en el código.
- Cada filtro se envía como parámetro (`CAST(:ini AS date)`, `CAST(:cat AS text)`, etc.), y "Todas" se convierte en NULL, que las funciones tratan como sin filtro.
- Las consultas se guardan en caché diez minutos con `st.cache_data`.
- El rango de fechas por defecto es enero 2017 a agosto 2018, los meses completos del dataset.

Ejecución.

```
$env:DB_URL = "postgresql+psycopg2://postgres:CLAVE@localhost:5432/olist_dm"
py -m streamlit run app.py
```

Evidencia: sección Ventas (filtros por defecto).



Evidencia: sección Logística y satisfacción (filtros por defecto).



Evidencia: un filtro modificado (por ejemplo Estado = SP) y cómo cambian los resultados.



Resultado relevante para la decisión (se completa al ver los datos reales en la sección de Logística).

```
PS C:\Users\dave> & "C:\Program Files\PostgreSQL\18\bin\psql.exe" -U postgre
n_satisfaccion_vs_retraso('2017-01-01','2018-08-31'); SELECT * FROM dm.fn_ta
LIMIT 5;"
Contraseña para usuario postgres:

   puntualidad    | pedidos | puntaje_promedio
------------------+---------+------------------
 A tiempo         |   89182 |             4.29
 Entrega tard├¡a |    6378 |             2.27
(2 filas)


 estado_cliente | pedidos_entregados | pct_tardios
----------------+--------------------+-------------
 AL             |                396 |       21.46
 MA             |                716 |       17.46
 SE             |                333 |       15.32
 PI             |                475 |       13.89
 CE             |               1275 |       13.80
(5 filas)


PS C:\Users\dave>
```

10. Uso de Inteligencia Artificial

Usé Claude como apoyo en tres momentos clave del proyecto, y en cada uno verifiqué el resultado ejecutándolo en mi propio equipo. Puedo explicar, modificar y justificar cada solución.

Uso 1. Error al cargar los datos con el ETL

Aspecto	Detalle
Problema	Al ejecutar `02_etl_cargar.py`, la lectura y limpieza funcionaron, pero la carga falló con `UnicodeDecodeError:`

Aspecto	Detalle
	`'utf-8' codec can't decode byte 0xf3`.
Prompt	Pegué la salida completa de PowerShell con el error.
Solución propuesta por la IA	El error de codificación era un síntoma: PostgreSQL, en Windows en español, respondía "autenticación fallida" con una ó que psycopg2 no sabe leer. La causa real era que dejé `TU_CLAVE` en la cadena de conexión.
Qué utilicé o modifiqué	Reemplacé `TU_CLAVE` por mi contraseña real en la variable `DB_URL`.
Cómo comprobé	El ETL terminó y mostró `Ítems en CSV: 112,650 | filas en fact_ventas: 112,650`.

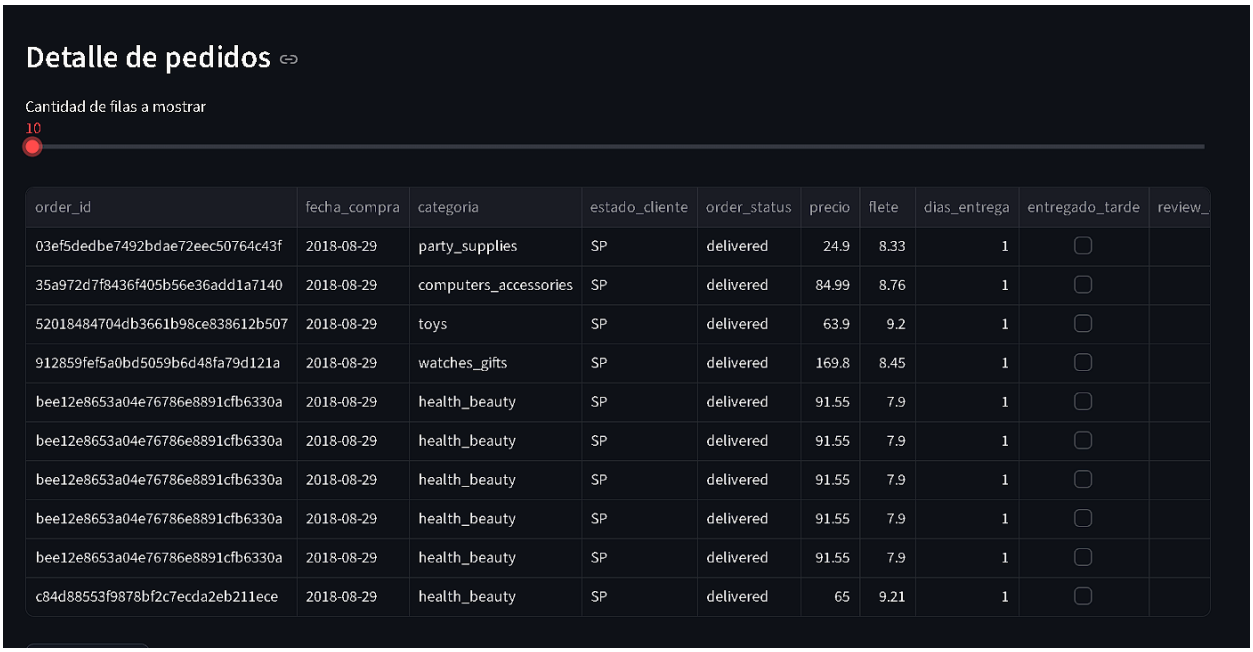
Uso 2. Funciones SQL que no encontraban la vista

Aspecto	Detalle
Problema	Al probar `fn_total_ingresos()`, PostgreSQL respondió `no existe la relación vw_ventas_detalle`.
Prompt	Pegué el error completo de `psql`.
Solución propuesta por la IA	Dos causas: (1) al recrear las tablas con `DROP ... CASCADE` se habían borrado las vistas, y (2) las funciones llaman a la vista sin esquema, por lo que una conexión nueva no la encuentra. Propuso ejecutar de nuevo `04_vistas_funciones.sql` y fijar `ALTER DATABASE olist_dm SET search_path TO dm, public;`.
Qué utilicé o modifiqué	Ejecuté ambos comandos en ese orden.
Cómo comprobé	La consulta de los 5 KPI devolvió valores, y los ingresos (13,494,400.74) coinciden con el total del CSV menos cancelados y no disponibles: 13,591,643.70 − 95,235.27 − 2,007.69.

Uso 3. Diseño de la aplicación Streamlit

Aspecto	Detalle
Problema	Necesitaba una aplicación con los requisitos del enunciado (2 secciones, 3 KPI, 3 filtros, 2 gráficos, tabla de detalle) que usara solo mis funciones SQL.
Prompt	Pasé el enunciado del Entregable 3 y mis scripts del

Aspecto	Detalle
	Entregable 2 y pregunté cómo construir la aplicación.
Solución propuesta por la IA	Un `app.py` con dos secciones, filtros globales enviados como parámetros a las funciones `dm.fn_*`, conexión por `DB_URL` y caché de consultas.
Qué utilicé o modifiqué	Usé el `app.py` propuesto por la IA y adapté la conexión a mi entorno: definí `DB_URL` con mi usuario, contraseña y el puerto 5432 de PostgreSQL 18, y lo ejecuté con `py -m streamlit run app.py`, porque los programas de Python no están en el PATH de mi equipo. Dejé el rango de fechas por defecto en enero de 2017 a agosto de 2018, que son los meses completos del dataset.
Cómo comprobé	Cambié los filtros y los KPI cambiaron; por ejemplo, con el estado en SP los ingresos bajaron de R$ 13,449,530 a R$ 5,152,573. Además, comparé el gráfico de satisfacción de la app con la consulta directa `fn_satisfaccion_vs_retraso` en psql: ambos dan 4.29 para entregas a tiempo y 2.27 para entregas tardías. Eso confirma que la aplicación muestra lo que calcula la base de datos.



order_id	fecha_compra	categoria	estado_cliente	order_status	precio	flete	dias_entrega	entregado_tarde	review_
03ef5dedbe7492bdae72eec50764c43f	2018-08-29	party_supplies	SP	delivered	24.9	8.33	1	☐	
35a972d7f8436f405b56e36add1a7140	2018-08-29	computers_accessories	SP	delivered	84.99	8.76	1	☐	
52018484704db3661b98ce838612b507	2018-08-29	toys	SP	delivered	63.9	9.2	1	☐	
912859fef5a0bd5059b6d48fa79d121a	2018-08-29	watches_gifts	SP	delivered	169.8	8.45	1	☐	
bee12e8653a04e76786e8891cfb6330a	2018-08-29	health_beauty	SP	delivered	91.55	7.9	1	☐	
bee12e8653a04e76786e8891cfb6330a	2018-08-29	health_beauty	SP	delivered	91.55	7.9	1	☐	
bee12e8653a04e76786e8891cfb6330a	2018-08-29	health_beauty	SP	delivered	91.55	7.9	1	☐	
bee12e8653a04e76786e8891cfb6330a	2018-08-29	health_beauty	SP	delivered	91.55	7.9	1	☐	
bee12e8653a04e76786e8891cfb6330a	2018-08-29	health_beauty	SP	delivered	91.55	7.9	1	☐	
c84d88553f9878bf2c7ecda2eb211ece	2018-08-29	health_beauty	SP	delivered	65	9.21	1	☐	


Descargar CSV
Consultas SQL de esta sección
SELECT dm.fn_tasa_entregas_tardias(ini, fin, categoria, uf);
SELECT dm.fn_tiempo_promedio_entrega(ini, fin, categoria, uf);
SELECT dm.fn_satisfaccion_promedio(ini, fin, categoria, uf);
SELECT * FROM dm.fn_tardias_por_estado(ini, fin, categoria);
SELECT * FROM dm.fn_satisfaccion_vs_retraso(ini, fin, categoria, uf);
SELECT * FROM dm.fn_detalle_pedidos(ini, fin, categoria, uf, limite);